



深圳大学  
Shenzhen University

# 互联网编程

## 第6章

## HTTP

当你在浏览器的地址栏输入例如:<http://www1.szu.edu.cn>, 回车后都发生了什么?

深圳大学计算机与软件学院

# 主要内容

- 1. HTTP协议
- 2. HTTP连接会话与会话保存
- 3. HTTP请求与响应
- 4. HTTP方法
- 5. Cookie



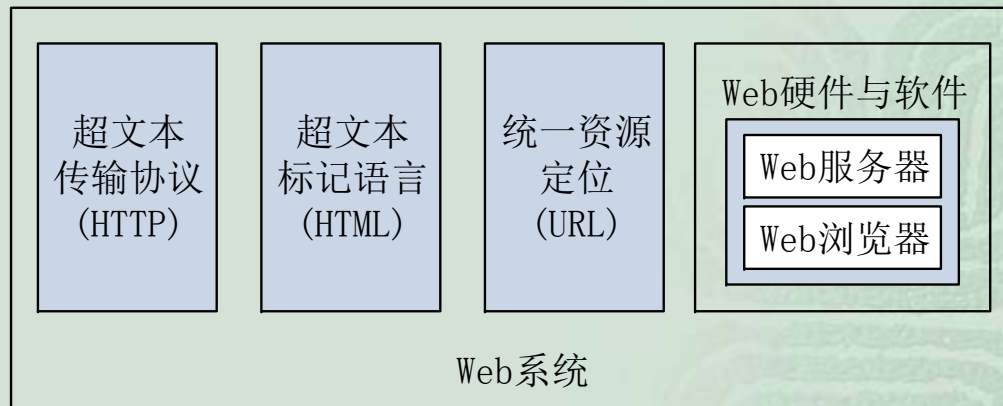
# 1.HTTP标准

- HTTP: **H**ypertext **T**ransfer **P**rotocol,超文本传输协议
- 定义了web客户端如何与服务器对话，数据如何从服务器传回客户端
- 可以传输 TIFF图片/Msword文档、Windows的.exe文件等任何可以用字节表示的东西。



# HTTP协议

- Web浏览器<->Web服务器之间通信的标准协议
  1. 客户端连接服务器80端口;
  2. 客户端发送请求
  3. 服务器发送响应
  4. 服务器关闭连接
- HTTP1.1中，第2、3步骤可以反复多次
  - 因为建立连接需要较高代价，为每个请求建立一个链接代价太高



# HTTP的工作机制

## 无状态协议的概念

- HTTP使用面向连接的TCP协议，客户端Web浏览器要与Web服务器之间建立一个TCP连接。
- TCP连接建立后，浏览器进程发送HTTP请求报文，并接收应答报文。
- Web服务器接收HTTP请求报文，并发送应答报文。
- TCP提供可靠服务保证客户进程发送的HTTP请求正确到达服务器端。服务器进程发送HTTP应答报文也正确达到客户端。
- Web服务器发送HTTP应答报文时，不保存浏览器的任何请求状态信息（属于无状态协议）
- HTTP是无状态协议





# HTTP的工作机制

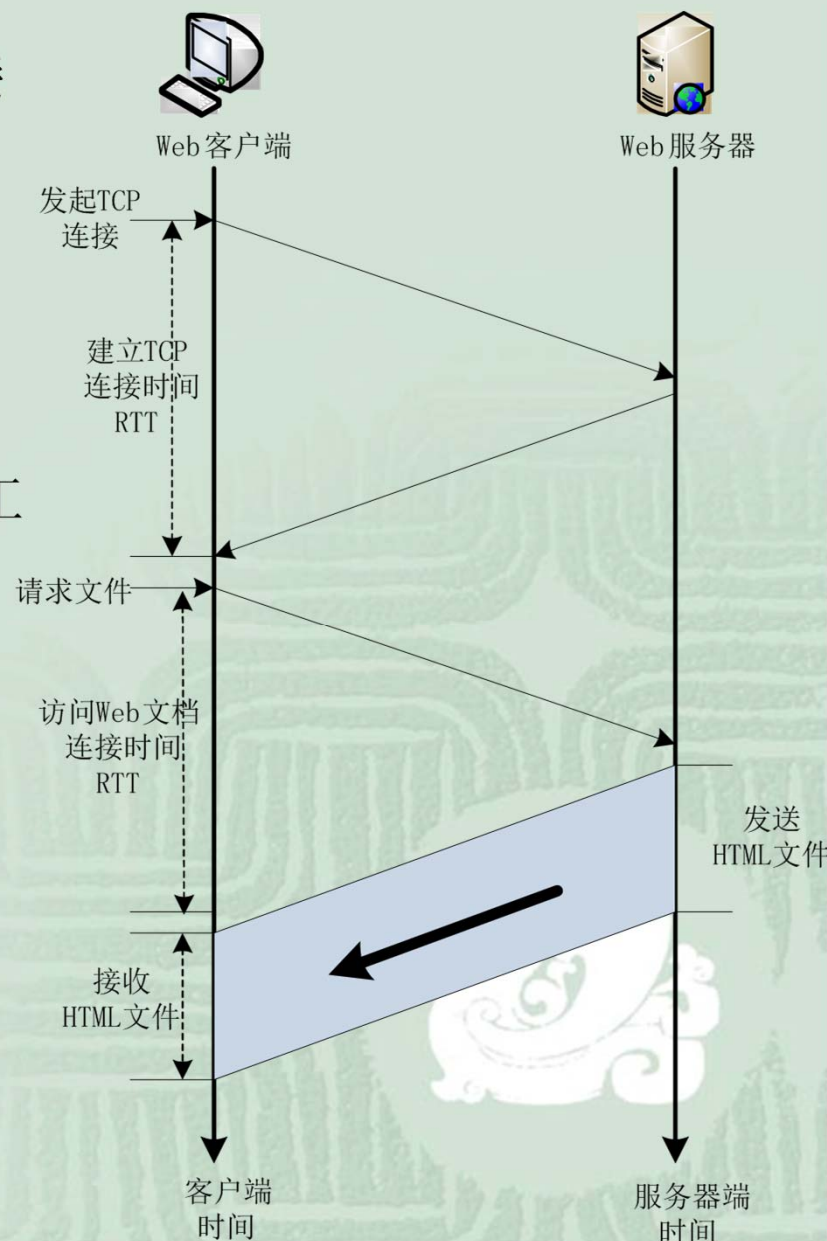
HTTP协议两种状态是非持续连接 和 持续连接

非持续连接（HTTP1.0）

每次请求/响应都要建立一次TCP连接。

例如：一个网页包括一个HTML文件和105个  
GIF图像文件（106个对象），那么浏览器工  
作过程为：

缺点：必须为每个请求对象建立和维护一个新  
的TCP连接。



# HTTP的工作机制

## 持续连接（HTTP1.1）

- 持续连接时，服务器在发出响应后保持该TCP连接，相同的客户端进程与服务器端之间的后续报文都通过该连接传送。

例如：一个网页包括一个HTML文件和8个JPEG图像文件，所有请求与应答报文都通过一个持续TCP连接来传送。

## 工作方式：

- 非流水线

客户端只有在接收到前一个响应时才能发出新的请求

- 流水线

客户端在没有收到前一个响应时就发出新的请求



## 2.HTTP连接会话与会话保存

### ■ 会话概念

用户开一个浏览器，点击多个超链接，访问服务器多个web资源，然后关闭浏览器，整个过程称之为一个会话。

#### □ 有状态会话

一个同学来过教室，下次再来教室，我们会知道这个同学曾经来过，这称之为有状态会话。

### ■ 会话过程中的编程问题

每个用户在使用浏览器与服务器进行会话的过程中，不可避免各自会产生一些数据，程序要想办法为每个用户保存这些会话数据





# 保存会话数据的两种技术

## ■ Cookie

- Cookie是客户端技术，
- 程序把每个用户的数据以cookie的形式写给用户各自的浏览器。
- 当用户使用浏览器再去访问服务器中的web资源时，就会带着各自的数据去。
- 这样，web资源处理的就是用户各自的数据了。

## ■ Session

- Session是服务器端技术，
- web服务器在运行时可以为每一个用户的浏览器创建一个其独享的session对象，
- 由于session为用户浏览器独享，所以用户在访问服务器的web资源时，可以把各自的数据放在各自的session中，
- 当用户再去访问服务器中的其它web资源时，其它web资源再从用户各自的session中取出数据为用户服务。

### 3.HTTP请求与响应：客户端请求

- 请求包括：
  - 请求行：GET /szu.asp HTTP1.1
  - 包含元数据的**HTTP**首部
    - 格式 keyword: value
    - Keyword和value必须全为ASCII
    - 行以\r\n结尾
  - 空行
  - 主体

一个GET请求

GET /szu.asp HTTP/1.1

方法 资源路径 HTTP版本

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.8; rv:20.0)  
Gecko/20100101 Firefox/20.0

Host: www.szu.edu.cn

Connection: keep-alive

Accept-Language: zh-CN,en;q=0.5

Accept-Encoding: gzip, deflate

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,\*/\*;q=0.8

# 客户端请求

- 一个POST请求向服务器发送表单数据:

```
POST /cgi-bin/register.pl HTTP 1.0
Date: Sun, 27 Apr 2013 12:32:36
Host: www.cafeaulait.org
Content-type: application/x-www-form-urlencoded
Content-length: 54

username=Elliotte+Harold&email=elharo%40ibiblio.org
```



# 客户端请求

- 一个上传Atom文档的PUT请求:

```
PUT /blog/software-development/the-power-of-pomodoros/ HTTP/1.1
Host: elharo.com
User-Agent: AtomMaker/1.0
Authorization: Basic ZGFmZnk6c2VjZXJldA==
Content-Type: application/atom+xml;type=entry
Content-Length: 322
```

```
<?xml version="1.0"?>
<entry xmlns="http://www.w3.org/2005/Atom">
  <title>The Power of Pomodoros</title>
  <id>urn:uuid:101a41a6-722b-4d9b-8afb-ccfb01d77499</id>

  <updated>2013-02-22T19:40:52Z</updated>
  <author><name>Elliotte Harold</name></author>
  <content>I hadn't paid much attention to Pomodoro...</content>
</entry>
```



# 客户端请求

- User-Agent: 浏览器信息
  - ▣ 从Netscape开始的历史
- Host: 主机名
  - ▣ 既然已经建立连接，为何还要主机名？
- Accpet: text/html,application/...
  - ▣ MIME类型：类型/子类型
  - ▣ text/\* 文本
  - ▣ image/\* 图像
  - ▣ application/\* 二进制
  - ▣ application/x-\* 自定义二进制
  - ▣ 所有MIME定义 <http://www.iana.org/assignments/media-types/media-types.xhtml>





## 思考问题:

Accept: text/html, text/plain, image/gif, image/jpeg

什么含义?

表示客户端可处理文本类型的html文件、plain纯文本文件，图像类型的gif文件和jpeg文件。



# 客户端请求编程

```
Socket socket= new Socket("www.szu.edu.cn",80);  
DataOutputStream out= new DataOutputStream( socket.getOutputStream());  
BufferedReader in = new BufferedReader(new  
InputStreamReader(socket.getInputStream()));  
  
String s = "GET " + url.getPath() + " HTTP/1.1\r\n";  
...  
s += "\r\n";  
out.writeBytes(s);
```

演示讲解HTTPGet.java



```
HTTPGet.java
3 import java.io.*;
4 import java.net.*;
5 public class HTTPGet{
6     public static void main(String args[]){
7         try{
8             URL url = new URL(args[0]);
9             //打印输入信息
10            System.out.println("Host:" + url.getHost());
11            System.out.println("Path:" + url.getPath());
12
13            Socket socket= new Socket(url.getHost(),80);
14            DataOutputStream out= new DataOutputStream( socket.getOutputStream());
15            BufferedReader in = new BufferedReader(
16                new InputStreamReader(socket.getInputStream()));
17
18            //发送HTTP请求
19            String s = "GET " + url.getPath() + " HTTP/1.1\r\n";
20            s += "User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.8; rv:20.0) Gecko/201001
21            s += "Host: " + url.getHost() + "\r\n";
22            s += "Connection: close\r\n";
23            s += "Accept-Language: zh-CN,en;q=0.5\r\n";
24            s += "Accept-Encoding: gzip, deflate\r\n";
25            s += "Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n";
26            s += "\r\n";
27
28            System.out.println(s);
```

```
29
30     out.writeBytes(s);
31     out.flush();
32
33     while((s=in.readLine())!=null){
34         System.out.println(s);
35     }
36     socket.close();
37 }
38 catch (MalformedURLException ex) {
39     System.err.println(args[0] + " is not a parseable URL");
40 }
41 catch(IOException e){
42     System.err.println(e);
43 }
44 }
45
```



## 客户端请求编程 及成功响应例子

请求

```
GET /szu.asp HTTP/1.1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.8; rv:20.0) Gecko/20100101 Firefox/20.0
Host: www.szu.edu.cn
Connection: keep-alive
Accept-Language: zh-CN,en;q=0.5
Accept-Encoding: gzip, deflate
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
```

```
HTTP/1.1 200 OK
Date: Sun, 27 Mar 2016 12:17:32 GMT
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
Content-Length: 21601
Content-Type: text/html
Set-Cookie: ASPSESSIONIDASARSQQQ=KBDKKEODPCJBAMOPMHNHJNH; path=/
Cache-control: private
```

## 成功响应例子

```
<head>  
<meta http-equiv="Content-Type" content="text/html; charset=gb2312">  
<meta name="keywords" content="深圳大学, 深大, 特区大学, Shenzhen University">  
<title>深圳大学内部网 | SHENZHEN UNIVERSITY</title>  
(http://www.szhu.edu.cn)
```



# 服务器端的HTTP响应

- 包含：状态行+ 信息头

- 状态行

- HTTP/1.1 200 OK

- 信息头

- key:value格式描述的响应
    - Date
    - Server
    - Connection
    - Content-Type
    - Content-length
    - 一个空行
    - Content



# HTTP1.1部分响应码

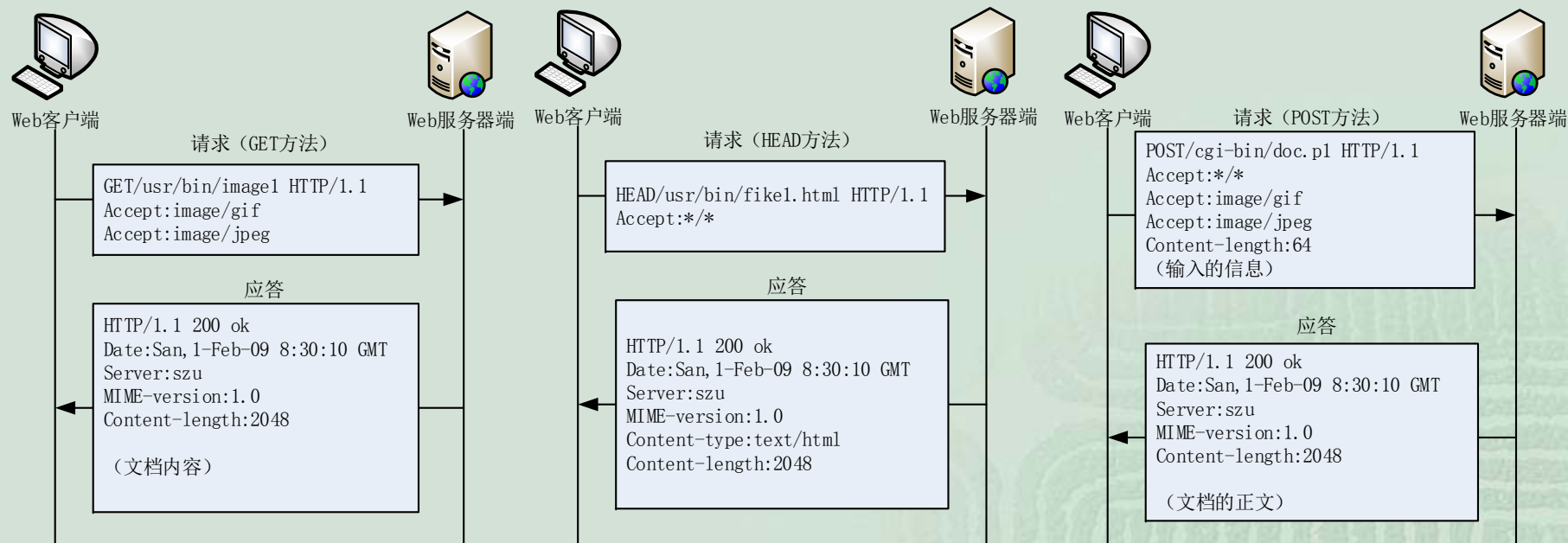
- 2xx请求成功
  - 200 OK
- 3xx 重定位或重定向
- 4xx 客户端错误
  - 400 Bad Request
  - 401 Unauthorized
  - 403 Forbidden
  - 404 Not Found
- 5xx 服务器错误
  - 500 Internal Server Error



# HTTP1.1响应码

| 代码  | 短语                  | 说明                       |
|-----|---------------------|--------------------------|
| 100 | Continue            | 请求的开始部分已经被接受，客户可以继续他的请求  |
| 101 | Switching           | 服务器同意客户的请求，切换到更新报头中定义的协议 |
| 200 | Ok                  | 请求成功                     |
| 201 | Created             | 新的URL被创建                 |
| 202 | Accepted            | 请求被接受，但还没有马上起作用          |
| 204 | No accepted         | 报文中没有内容                  |
| 301 | Multiple choices    | 所请求的URL指向多个资源            |
| 302 | Moved permanently   | 服务器已经不再使用所使用的URL         |
| 304 | Moved temporarily   | 所请求的URL已暂时地移走            |
| 400 | Bad request         | 在请求中有语法错误                |
| 401 | Unauthorized        | 请求缺乏适当的授权                |
| 403 | Forbidden           | 服务被拒绝                    |
| 404 | Not found           | 文档未发现                    |
| 405 | Method not allowed  | URL不支持                   |
| 406 | Not acceptable      | 所请求的格式不可接受               |
| 500 | Server error        | 服务器端出错                   |
| 501 | Not implemented     | 所请求的动作不能完成               |
| 503 | Service unavailable | 服务器暂时不可使用，但以后可能接受请求      |

# 请求和应答报文的交互过程



## 4.HTTP方法

- **4个方法：**GET、POST、PUT、DELETE 标识可完成的操作。
- 方法各有特定的语义，编程时必须遵循语义。
- **GET方法：**用于获取URL所标识的一个资源的表示。

```
GET /register.jsp?username=elliotte&email=elharo%40ibiblio.org HTTP/1.1  
Host: www.cafeaulait.org
```

- **应用场景：**用于非提交的动作。
  - 如浏览一个静态Web页面。
  - 如在购物车里增加一个商品。



# POST方法

- **POST方法**：将资源的一个表示上传到已知URL的服务器，但没有指定服务器如何处理这个新提供的资源。
  - 应用场景：用于不能重复的不安全的操作场景，常用于提交某个东西的动作。
    - 如一个交易中的下订单，此动作完成了一个提交。
    - 例子：一个POST请求，向服务器发送表单数据。

POST /register.jsp HTTP/1.1

Host: www.cafeaulait.org

Content-type: application/x-www-form-urlencoded

Content-length: 54

username=elliotte&email=elharo%40ibiblio.org

帐号:  @email.szu.edu.cn

密码:

[忘记密码?](#)

 正在使用https方式登录





# DELETE和PUT方法

PUT用于上传资源、DELETE用于删除资源

- **DELETE方法**: 从一个指定的URL删除一个资源。
- **PUT方法**: 将资源的一个表示上传到已知URL的服务器。
  - ▣ PUT请求例子，上传Atom文档的put请求:

**PUT /blog/software-development/the-power-of-pomodoros/ HTTP/1.1**

**Host: elharo.com**

**User-Agent: AtomMaker/1.0**

**Authorization: Basic ZGFmZnk6c2VjZXJldA==**

**Content-Type: application/atom+xml;type=entry**

**Content-Length: 322**

```
<?xml version="1.0"?>
<entry xmlns="http://www.w3.org/2005/Atom">
  <title>The Power of Pomodoros</title>
  <id>urn:uuid:101a41a6-722b-4d9b-8afb-ccfb01d77499</id>
  <updated>2013-02-22T19:40:52Z</updated>
  <author><name>Elliote Harold</name></author>
  <content>I hadn't paid much attention to Pomodoro...</content>
</entry>
```



## HEAD方法

- **HEAD方法**: 仅返回资源信息头, 不返回资源内容  
可用于检查资源是否有更新

```
java HTTPGet http://www.szu.edu.cn/images/index/s20151.jpg
```

```
HTTP/1.1 200 OK
Content-Length: 27359
Content-Type: image/jpeg
Last-Modified: Wed, 30 Dec 2015 03:03:54 GMT
Accept-Ranges: bytes
ETag: "5c22ceb7ae42d11:e2ad"
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
Date: Sun, 27 Mar 2016 13:33:12 GMT
Connection: close
```



## 5.Cookie

- **会话跟踪编程需求**:在程序中，会话跟踪是很重要的事情。
  - 理论上，一个用户的所有请求操作都应该属于同一个会话，而另一个用户的所有请求操作则应该属于另一个会话，二者不能混淆。
  - 例如，用户A在超市购买的任何商品都应该放在A的购物车内，不论是用户A什么时间购买的，这都是属于同一个会话的，不能放入用户B或用户C的购物车内，这不属于同一个会话。
- Web应用程序是使用HTTP协议传输数据的。**HTTP协议是无状态的协议**。一旦数据交换完毕，客户端与服务器端的连接就会关闭，再次交换数据需要建立新的连接。这就意味着**服务器无法从连接上跟踪会话**。
  - 即用户A购买了一件商品放入购物车内，当再次购买商品时服务器已经无法判断该购买行为是属于用户A的会话还是用户B的会话了。
- 要跟踪该会话，必须引入一种机制。
  - 在Session出现之前，基本上所有的网站都采用Cookie来跟踪会话

## 5.Cookie

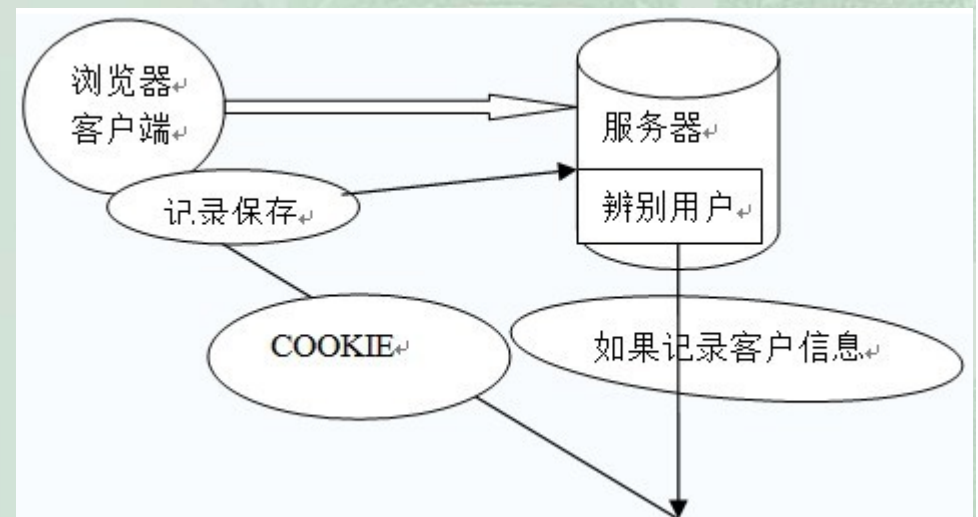
- **Cookie**:意为“甜饼”，是由W3C组织提出，最早由Netscape社区发展的一种机制。目前Cookie已经成为标准，所有的主流浏览器如IE、Netscape、Firefox、Opera等都支持Cookie。
- Cookie技术是客户端的解决方案，Cookie就是由服务器发给客户端的特殊信息，这些信息以文本文件的方式存放在客户端，然后客户端每次向服务器发送请求的时候都会带上这些特殊的信息。身份信息
- Cookie 是您访问过的网站创建的文件，用于存储浏览信息，例如您的网站偏好设置或个人资料信息。

<http://www.szu.edu.cn/szu.asp>的返回

```
HTTP/1.1 200 OK
Connection: close
Date: Sun, 27 Mar 2016 13:41:28 GMT
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
Content-Length: 21601
Content-Type: text/html
Set-Cookie: ASPSESSIONIDASARSQQQ=FLFMKEODONDNPGAPEIIAGNOD; path=/
Cache-control: private
```

# Cookie工作原理

- HTTP是一种无状态的协议，服务器单从网络连接上无从知道客户身份。
- 怎么办呢？（Cookie的工作原理）
  - 就给客户端们颁发一个通行证，每人一个，无论谁访问都必须携带自己通行证。这样服务器就能从通行证上确认客户身份了。
  - 客户端请求服务器，如果服务器需要记录该用户状态，就使用response向客户端浏览器颁发一个Cookie。客户端浏览器会把Cookie保存起来。当浏览器再请求该网站时，浏览器把请求的网址连同该Cookie一同提交给服务器。服务器检查该Cookie，以此来辨认用户状态。服务器还可以根据需要修改Cookie的内容。
- Cookie实际上是一小段的文本信息。





## 浏览器对Cookie功能的支持

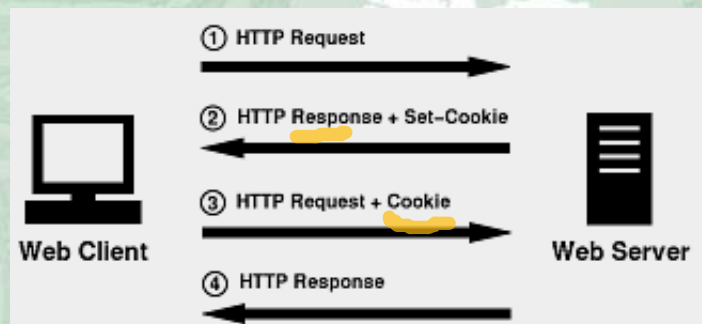
- Cookie功能需要浏览器的支持。
- 如果浏览器不支持Cookie（如大部分手机中的浏览器）或者把Cookie禁用了，Cookie功能就会失效。
- 不同的浏览器采用不同的方式保存Cookie。
  - IE浏览器会在“C:\Documents and Settings\你的用户名\Cookies”文件夹下以文本文件形式保存，一个文本文件保存一个Cookie。





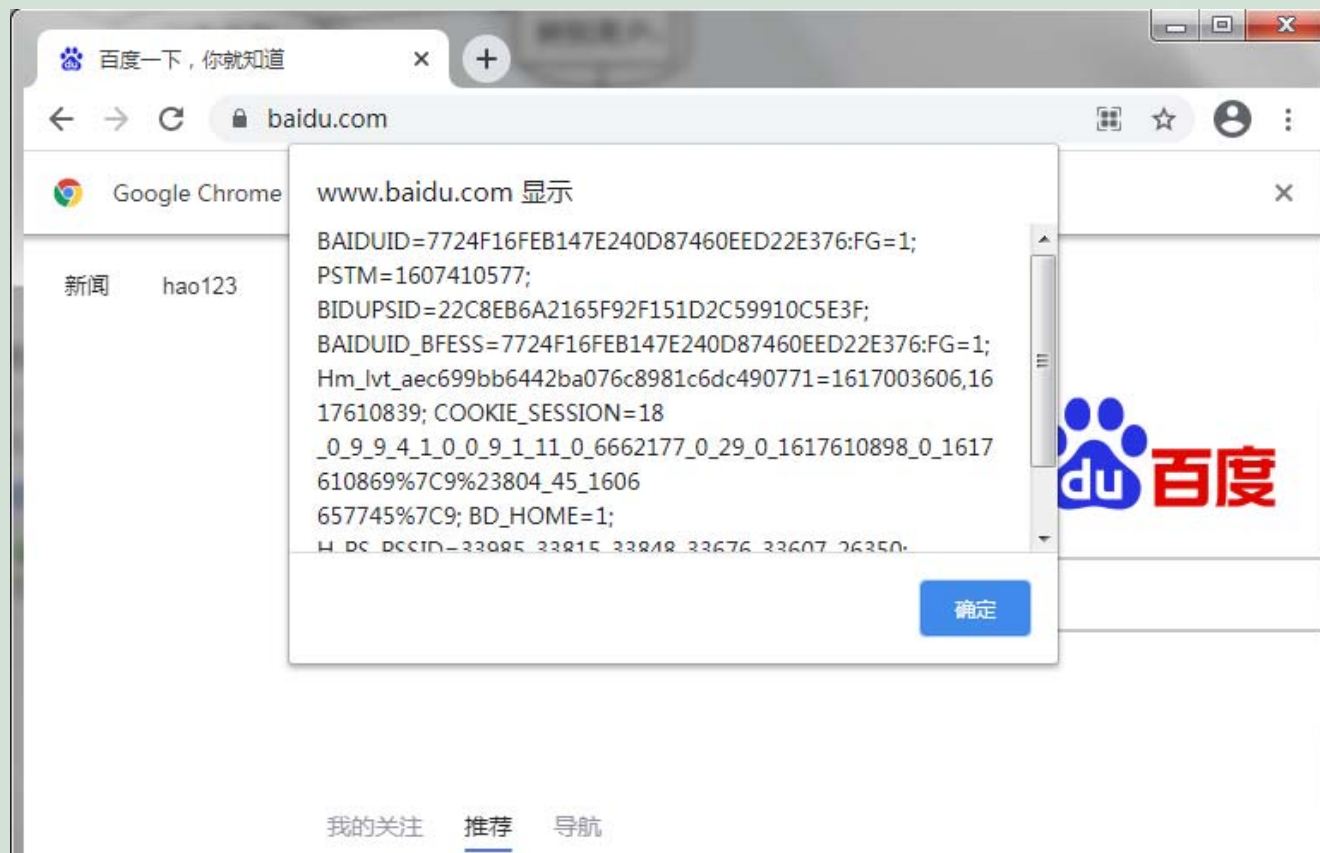
# Cookie详细工作过程

- 当用户使用浏览器访问一个支持Cookie的网站的时候，用户会提供包括用户名在内的个人信息并且提交至服务器；
- 服务器在向客户端回传相应的超文本的同时也会发回这些个人信息，当然这些信息并不是存放在HTTP响应体（Response Body）中的，而是存放于HTTP响应头（Response Header）；
- 当客户端浏览器接收到来自服务器的响应之后，浏览器会将这些信息存放在一个统一的位置，Windows操作系统可以从：[系统盘]:\Documents and Settings[用户名]\Cookies目录中找到存储的Cookie；  
Cookie都存在于请求/响应头部
- 客户端再向服务器发送请求的时候，都会把相应的Cookie再次发回至服务器。而这次，Cookie信息则存放在HTTP请求头（Request Header）了。
- 有了Cookie技术实现，服务器在接收到来自客户端浏览器的请求之后，就能够通过分析存放于请求头的Cookie得到客户端特有的信息，从而动态生成与该客户端相对应的内容。
  - 通常，我们可以从很多网站的登录界面中看到“请记住我”这样的选项，如果你勾选了它之后再登录，在下一次访问该网站的时候就不需要进行重复而繁琐的登录动作了，而这个功能就是通过Cookie实现的。



## 查看某个网站颁发的Cookie

- 打开某个网站，在浏览器地址栏输入javascript:alert (document.cookie)就可以查看该网站颁发的Cookie。（需要有网才能查看）
- JavaScript脚本会弹出一个对话框显示本网站颁发的所有Cookie的内容
- 图中弹出的对话框中显示的为Baidu网站的Cookie。其中第一行BAIDUID记录的就是运行者我的身份，只是Baidu使用特殊的方法将Cookie信息加密了。



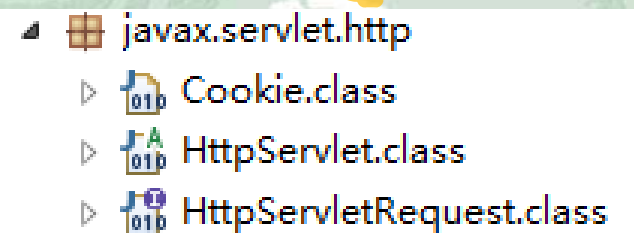
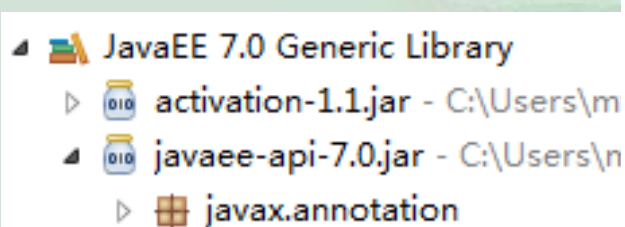
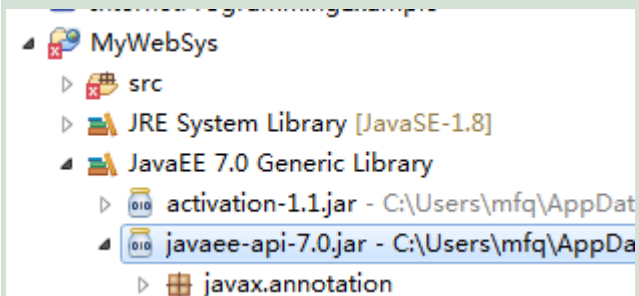
# Cookie的不可跨域名性

一个网站一个Cookie，不可重用

- 很多网站都会使用Cookie。例如，Google会向客户端颁发Cookie，Baidu也会向客户端颁发Cookie。
- 那浏览器访问Google会不会也携带上Baidu颁发的Cookie呢？或者Google能不能修改Baidu颁发的Cookie呢？
- 答案是否定的。Cookie具有不可跨域名性。根据Cookie规范，浏览器访问Google只会携带Google的Cookie，而不会携带Baidu的Cookie。Google也只能操作Google的Cookie，而不能操作Baidu的Cookie。
- **Cookie在客户端是由浏览器来管理的。**
  - 浏览器能够保证Google只会操作Google的Cookie而不会操作Baidu的Cookie，从而保证用户的隐私安全。
  - 浏览器判断一个网站是否能操作另一个网站Cookie的依据是域名。
  - Google与Baidu的域名不一样，因此Google不能操作Baidu的Cookie。
  - 例子：网站images.google.com与网站www.google.com同属于Google，但是域名不一样，二者同样不能互相操作彼此的Cookie
  - 特殊处理：用户登录网站www.google.com之后会发现访问images.google.com时登录信息仍然有效，而普通的Cookie是做不到的。这是因为Google做了特殊处理。

# Cookie编程

- Java中把Cookie封装成了`javax.servlet.http.Cookie`类（在`servlet-api.jar`包中）。
- 每个Cookie都是该Cookie类的对象。
- 服务器通过操作Cookie类对象对客户端Cookie进行操作。
  - 通过`request.getCookies()`获取客户端提交的所有Cookie（以`Cookies`数组形式返回），
  - 通过`response.addCookie(Cookie cookie)`向客户端设置Cookie。
- Cookie对象使用`key-value`属性对的形式保存用户状态，
  - 一个Cookie对象保存一个属性对，一个request或者response同时使用多个Cookie。
- 因为Cookie类位于包`javax.servlet.http.*`下面，所以JSP中不需要import该类。





# Cookie编码

- **ASCII编码**，保存英文：只能是非空白符的ASCII文本，不包含逗号或分号。
- **Unicode编码**，保存中文：中文与英文字符不同，中文属于Unicode字符，在内存中占4个字节，英文属于ASCII字符，内存中只占2个字节。Cookie中使用Unicode字符时需要对Unicode字符进行编码，否则会乱码。
  - Cookie中保存中文只能编码。一般使用UTF-8编码即可。不推荐使用GBK等中文编码，因为浏览器不一定支持，而且JavaScript也不支持GBK编码。
- **BASE64编码**，保存二进制图片：Cookie不仅可以与ASCII字符与Unicode字符，还可以使用二进制数据。
  - 例如在Cookie中使用数字证书，提供安全度。使用二进制数据时也需要进行编码。
  - 注意：Cookie中可以存储二进制内容，并不实用。由于浏览器每次请求服务器都会携带Cookie，因此Cookie内容不宜过多，否则影响速度。Cookie的内容应该少而精。

# Cookie

- Cookie作用域

set-Cookie: domain=.szu.edu.cn

- 受路径限制的作用域

set-Cookie: path=/home ;domain=.szu.edu.cn

- 其它属性设置

set-Cookie: user=elharo; expires=Wed, 21-Dec-2015 15:23:00 GMT

set-Cookie: user=elharo; Max-Age=3600

(3600秒，即1小时后过期)





# CookieManager类

- 启用**cookie**:编程时用**URL**类连接**HTTP**服务器, 希望存储和获取**HTTP**服务器发送的**cookie**, 必须在程序中启用**cookie**。

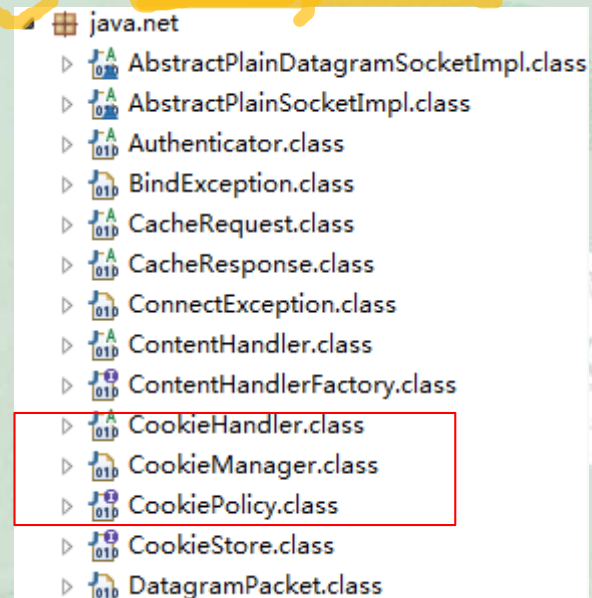
```
CookieManager manager = new CookieManager();  
CookieHandler.setDefault(manager);
```

- 接收策略

CookiePolicy.ACCEPT\_ALL 接受所有cookie

CookiePolicy.ACCEPT\_NONE不接受任何cookie

CookiePolicy.ACCEPT\_ORIGINAL\_SERVER只接受第一方cookie



# CookieManager类

- 阻塞第三方cookie，只接受第一方：

```
CookieManager manager = new CookieManager();  
Manager.setCookiePolicy(CookiePolicy.ACCEPT_ORIGINAL_SERVER);  
CookieHandler.setDefault(manager);
```

- 编程自定义CookiePolicy
- 通过实现CookiePolicy接口，并覆盖shouldAccept()方法：  
public boolean shouldAccept(URI uri, HttpCookie cookie)
- 下面给出自定义CookiePolicy类编程例子。



# 自定义CookiePolicy编程例子

- 编程自定义一个简单的cookie策略类,阻塞来自.gov域的cookie,接受所有其他域的cookie.
- Ex6-1 NoGovernmentCookies.java

```
import java.net.*;
public class NoGovernmentCookies implements CookiePolicy {
    @Override
    public boolean shouldAccept(URI uri, HttpCookie cookie) {
        if (uri.getAuthority().toLowerCase().endsWith(".gov")
            || cookie.getDomain().toLowerCase().endsWith(".gov")) {
            return false;
        }
        return true;
    }
}
```

# CookieStore类

- 有时需要在应用退出时，把cookie库保存在磁盘上，下一次启动应用时再加载这些cookie.
- 使用CookieManager类的getCookieStore()方法，cookieManger会保存它的cookie:

```
CookieStore store = manager.getCookieStore();
```

- **CookieStore**类提供了方法用于增加、删除和列出cookie，是你能控制在正常HTTP请求和响应流之外发送的 **Cookie**:
  - ▣ Public void add(URI uri, HttpCookie cookie)
  - ▣ Public List<HttpCookie> get(URI uri)
  - ▣ Public List<HttpCookie> getCookies()
  - ▣ Public List<URI> getURIs()
  - ▣ Public boolean remove(URI uri, HttpCookie cookie)
  - ▣ Public boolean removeAll()
- Cookie store库中的每个cookie都封装在一个HTTP cookie对象中，
- HttpCookie类提供了一些方法用于检查cookie的属性。

谢谢

